

ASSIGNMENT-1 MACHINE LEARNING

CHANDAN VARMA(G24AIT2017)

&

MANVENDRA PRATAP SINGH(G24AIT2163)

INDIAN INSTITUTE OF TECHNOLOGY-JODHPUR

Course Number - CSL7XX0

Instructor - DR.HARSHAL AKOLEKAR

Date *25-FEBRUARY-2025*



Solution 1: Data Preprocessing

1. Assign a type to each of the following features

a) Model → **Nominal**; b) Type → **Nominal**; c) **Max. Price** → **Ratio**; d) Airbags → **Ordinal**

2. Write a function to handle the missing values in the dataset (e.g., any NA, NaN values).

- Fills categorical columns with the most frequent value (mode).
- Fills numeric columns with their mean.

function ⇒ *Solution 1.2 (.ipynb)*

3. Write a function to reduce noise (any error in the feature) in individual attributes.

- Filtering outliers in numerical columns using the Interquartile Range (IQR)
- Standardizing categorical values by trimming spaces and unifying case format
- Rounding floating-point numbers to maintain consistency

function ⇒ *Solution 1.3 (.ipynb)*

4. Write a function to encode all the categorical features in the dataset according to the type of variable jointly.

- One-Hot Encoding: Converts nominal categorical features into binary (0/1) columns
- Ordinal Encoding: Assigns ordered numerical values to ordinal categorical features

function ⇒ *Solution 1.4 (.ipynb)*

5. Write a function to normalize / scale the features either individually or jointly.

Scaling techniques:

- Min-Max Scaling(Normalization): if method="minmax", Rescales values between 0 and 1
- Standardization(Z-score Scaling): method="minmax", Centers data around mean = 0, SD = 1

Scaling Modes:

1. Individually → Each column scaled separately: (individual == True)
 - `scaler.fit_transform(df_scaled[numeric_cols])` is applied column by column
2. Jointly → All features scaled together: (individual == False)
 - `scaler.fit_transform(df_scaled)` is applied to the entire DataFrame at once.

function ⇒ *Solution 1.5 (.ipynb)*

6. Write a function to create a random split of the data into train, validation and test sets in the ratio of [70:20:10]

we can use scikit-learn's `train_test_split` function in two steps:

1. First, split the dataset into train (70%) and temp (30%).
2. Then, split temp (30%) into validation (20%) and test (10%).

function ⇒ *Solution 1.6 (.ipynb)*

Solution 2a: Linear Regression Task

Implementation:

- **Data Preparation:** Load the dataset and split it into training (80%) and testing (20%) sets.
- **Sklearn Linear Regression:** Make predictions by using built-in **sklearn** LinearRegression method. (red line)
- **Manual Linear Regression:** Use the Normal Equation to compute regression coefficients and make predictions. (green dashed line)
- **Comparison & Visualization:** Plot the actual data and compare the prediction lines from both methods.

function ⇒ *Solution 2.a.3 (.ipynb)*

Observation:

- Comparing both in **graphs** the built-in `sklearn` **linear regression** and **Normal Equation** for linear regression the graph shows the similar results.
 - This confirms the correctness of the Normal Equation approach for calculating regression coefficients.
 - The close match between the two regression lines confirms that sklearn optimizes the same mathematical principles with less code, validating Linear Regression theory and the library's efficiency.
-

Solution 2b: Logistic Regression Task

Implementation:

- **Data Preparation:** Load the dataset and apply Label Encoding to convert categorical features into numeric values.
- **Dataset Splitting:** Split the data into 70% training and 30% testing sets.
- **Model Training:** Use **LogisticRegression** method from **sklearn** to train the Logistic Regression model.
- **Prediction & Evaluation:** Make predictions on the test set and evaluate performance using accuracy, confusion matrix, and classification report.

function ⇒ *Solution 2.b.3 (.ipynb)*

Observation

- From this we can say that logistic regression effectively classifies data and provides key performance metrics such as precision, accuracy, recall, and F1 score.
- The confusion matrix helps to analyze misclassifications, while high accuracy indicates a well-fitted model.
- Label encoding ensures that categorical data is used correctly in the model.

Solution 3: SVM

function $\Rightarrow$ *Solution 3 (.ipynb)*

Implementation Steps:

- **Load & Explore Dataset:** Loaded the dataset from Google Drive, checked its shape, and displayed sample records.
- **Feature Selection & Preprocessing:** Selected 'Age', 'Income', and 'CCAvg' as features and 'CreditCard' as the target. Handle missing values by filling with the median.
- **Data Splitting, Score and Report:** Split the dataset into 80% training and 20% testing. Predictions were made on the Test Data, Performance metrics viz. **Model Score, Confusion Matrix and Classification Report** were calculated for each C value and model's performance was evaluated.
- **GridSearchCV:-** Cross validation technique was used to find the best regularization value of C.
- **Model Training:**
 - Trained SVC models with different kernels (linear, polynomial, RBF, sigmoid) using various C values.
- **Evaluation:**
 - Analyzed classification reports and confusion matrices for Kernels comparison.
- **Kernel Comparison Result:-**

Sr. No.	Kernel	Best C	Degree	Gamma	Test Accuracy
1	Linear	0.1	-	-	0.481
2	RBF	100.0	-	0.01	0.598
3	Sigmoid	1.0	-	0.1	0.517
4	Polly	0.01	4	-	0.670

1. Linear Kernel: Performs best at C = 0.1 with a test accuracy of 0.481. Increasing C beyond this doesn't significantly improve performance.
2. RBF Kernel: Performs best at C = 100 and gamma = 0.01, achieving 0.598 test accuracy. RBF is better at capturing non-linear relationships.
3. Sigmoid Kernel: Performs best at C = 1 and gamma = 0.1, achieving 0.517 test accuracy.
4. Polynomial Kernel: Performs best at C = 0.01, degree = 4, achieving the highest accuracy (0.670) among all kernels. This suggests polynomial transformations fit the data better than other kernels.

- **Conclusion:-**

- > Polynomial Kernel (Degree = 4) is the best choice for this dataset.
- > RBF also performs well, but Polynomial gives better results.
- > Linear SVM is not a good fit, as the data is not linearly separable.
- > Choosing an appropriate C and gamma significantly impacts model performance.

Recommendation: If computational efficiency is a concern, RBF can be preferred over Poly since polynomial kernels with high degrees can be computationally expensive.

Q4: Decision Tree and Random Forest

function ⇒ *Solution 4 (.ipynb)*

Performing Decision Tree and Random Forest classification on the **Iris dataset**. Here are the steps we perform in our code:

Implementation:

1. **Load & Visualize the Dataset:** Load the Iris dataset and analyze feature distributions using histograms, pair plots, and class distribution plots.
2. **Encode Categorical Target Labels:** Convert species labels into numerical values using Label Encoding for model compatibility.
3. **Split Data into Training & Testing Sets:** Divide the dataset into 70% training and 30% testing sets to ensure a balanced evaluation.
4. **Train & Visualize the Decision Tree Model:** Build a Decision Tree classifier, train it on the dataset, and visualize the decision tree structure.
5. **Train & Tune the Random Forest Model:** Train a Random Forest classifier with multiple decision trees, and optionally tune hyperparameters (e.g., number of trees, depth, minimum samples split) to optimize performance.
6. **Model Evaluation & Comparison:** Evaluate both models using accuracy, confusion matrix, and classification reports, then compare their effectiveness.

Observation:

- Both the **Decision Tree** and **Random Forest** models achieved **100% accuracy**, indicating that the dataset is well-structured and easily separable.
- The **confusion matrices** show no misclassifications, meaning both models perfectly classified all samples across the three species.
- Since the dataset is relatively small and well-balanced, the Decision Tree did not overfit, performing as well as the Random Forest.
- In more complex datasets, **Random Forest typically generalizes better** by reducing variance, but in this case, both models performed equally well.
- This result suggests that for simple and well-separated datasets like Iris, a **Decision Tree can be as effective as a Random Forest**, making it a computationally efficient choice.